3(a). UML Class Diagram Changes

From Milestone 3 to Milestone 4, several major structural enhancements were made to the system, and these changes are reflected in the updated UML diagram. The most significant addition is the GameHistoryManager class, which introduces full undo/redo support through deep-copy state management. Implementing this feature required extending multiple classes, including Card, Player, PileSystem, and UNOModel, to perform deep cloning, ensuring fully isolated snapshots of game state. The UNOModel class expanded substantially to accommodate new gameplay mechanics, such as flip-side logic, wild-color handling, advanced draw rules, and save/load functionality. Milestone 4 also introduced a more robust event-driven communication pattern through the new UNOEvent class, allowing the model to deliver complete state updates to the view. Correspondingly, the UNOView and UNOController were extended to support new statuses, menu options, and UI behaviors. Finally, relationships in the UML were clarified using proper associations, aggregations, and compositions, improving overall structural clarity and adherence to MVC principles. These updates were essential for meeting the new functional requirements of Milestone 4 and strengthening the system's modularity and maintainability.

Key Changes Summarized

- **Added GameHistoryManager** to implement Undo/Redo with stack-based deep state storage.
- **Implemented deep cloning** across major classes to support reliable state restoration.
- **Expanded UNOModel** with new fields, flags, and methods for flip mechanics, wild rules, scoring, saving/loading, and event generation.
- **Introduced UNOEvent** to carry complete model state to the view, enabling detailed UI updates.
- **Enhanced UNOView and UNOController** with support for undo/redo, save/load, flip behavior, and new game statuses.
- **Clarified UML relationships**:
    - Composition for UNOModel → PileSystem and GameHistoryManager
    - Aggregation for UNOModel → Player and IView
    - Associations for controller/view/model interactions
- **Improved MVC separation** through event-driven updates instead of tightly coupled logic.